Javascript + Stimulus

Table des matières

1. Introduction 2
2. Installation 2
3. Pratique 2
Étape N°1 4
Étape N°2 4
Étape N°3 4
Étape N°4 5
Étape N°5 5
4. Aller plus loin 6

1. Introduction

Symfony a une approche bien pensée du JavaScript : plutôt que d'imposer un framework frontend lourd (React, Vue…), il propose une architecture légère appelée **AssetMapper + Stimulus**, intégrée via le bundle **Symfony UX**.

Pourquoi Stimulus ?

Le principe est simple : le serveur génère du HTML (comme toujours avec Symfony), et Stimulus vient **augmenter** ce HTML avec des comportements JavaScript. On parle d'approche *"HTML-first"*.

Contrairement à React ou Vue, Stimulus ne gère pas le rendu de l'interface — il se greffe sur le DOM existant. C'est pourquoi il s'intègre si naturellement avec Twig.

Stimulus s'articule autour de **5 concepts** :

Controller — C'est le cœur. Un fichier JS qui hérite de `Controller` (Stimulus). Il est associé à un élément HTML via l'attribut `data-controller="nom"`.

Targets — Des éléments HTML à l'intérieur du controller, déclarés avec `data-nom-target="monElement"`. Stimulus les expose automatiquement dans le JS via `this.monElementTarget`.

Actions — Des événements DOM déclarés directement en HTML : `data-action="click->monController#maMethode"`. Aucun `addEventListener` manuel.

Values — Un état réactif associé au controller. Déclaré via `data-nom-value-monAttribut="42"` et synchronisé automatiquement avec un callback quand il change.

Classes — Permet de gérer des classes CSS dynamiquement sans les coder en dur dans le JS.

2. Installation

Stimulus est déjà intégré si on a crée un projet avec le flag --webapp, mais si jamais vous en avez besoin pour autre chose :

```
php bin/console importmap:require @hotwired/stimulus
```

3. Pratique

Montez un nouveau projet appelé « **stimulus_demo** », avec le flag webapp. Pas de docker ici, on n'a pas besoin d'une BDD pour comprendre ce que ça va faire ☺

Puis on va se créer un contrôleur « **HomeController** » qui va intercépter l'URL « / » pour afficher le template **/home/index.html.twig** dans lequel on va aller coller ceci :

```
{% extends 'base.html.twig' %}

{% block title %}Hello HomeController!{% endblock %}

{% block body %}
<div
    data-controller="compteur"
    data-compteur-valeur-value="0"
    data-compteur-step-value="1">
    <button data-action="click->compteur#decrementer">-</button>

    <span data-compteur-target="affichage">0</span>

    <button data-action="click->compteur#incrementer">+</button>

    <p> Step: <input

        type="number"
        data-action="change->compteur#changerStep"
        value="1">
    </p>
</div>
{% endblock %}
```

Enfin, on va aller dans le dossier /assets/controllers et on va créer un fichier nommé **compteur_controller.js**

IL EST PRIMORDIAL QUE LE NOM DU FICHIER CORRESPONDE AU NOM DU CONTROLLEUR DANS L'ATTRIBUT DATA-CONTROLLER

C'est comme ça que Stimulus va faire le mapping. Voici le contenu de ce fichier à coller :

```
import { Controller } from '@hotwired/stimulus';

export default class extends Controller {

    static targets = ['affichage'];
    static values  = { valeur: Number, step: Number };

    valeurValueChanged() {
        this.affichageTarget.textContent = this.valeurValue;
    }

    incrementer() {
        this.valeurValue += this.stepValue;
    }

    decrementer() {
        this.valeurValue -= this.stepValue;
    }

    changerStep(event) {
        this.stepValue = parseInt(event.target.value) || 1;
    }
}
```

On a fini de copier, maintenant on va pouvoir détailler ce qui se passe dans l'ordre ☺

Étape N°1

Le fichier app.js charge le fichier « stimulus_bootstrap.js ». Sans ça, Stimulus n'existe même pas aux yeux de Symfony.

Étape N°2

Le fichier stimulus_bootstrap charge tous les fichiers contenus dans assets/controllers grâce à cette ligne :

```
const app = startStimulusApp();
```

C'est pour ça qu'on doit nommer nos attributs « data-controller » exactement comme les fichiers, c'est parce que Stimulus a cette façon de détecter les fichiers et de les faire correspondre à un élément HTML.

Étape N°3

Le contrôleur détermine tout le fonctionnement avec des règles de nommages **strictes** : tout ce qu'on lui passe dans le tableau **affichage** donne lieu à trois propriétés sur **this**, par exemple pour les **targets** (les éléments HTML qui seront ciblés) :

```
    static targets = ['affichage'];
```

Stimulus génère automatiquement **trois propriétés** pour chaque target déclarée :

Propriété	Type	Description
this.**affichage**Target	Element	Le premier élément trouvé
this.**affichage**Targets	Element[]	Tous les éléments correspondants
this.has**Affichage**Target	Boolean	true si au moins un existe

Pour les **values**, c'est la même logique :

```
    static values  = { valeur: Number, step: Number };
```

Pour chaque value, Stimulus génère **une propriété + un callback** :

Généré	Rôle
this.**valeur**Value	Getter/setter (lit et écrit la value)
this.**step**Value	Getter/setter
valeurValueChanged()	Callback appelé automatiquement à chaque changement
stepValueChanged()	Idem pour step

Étape N°4

Dans le HTML, enfin, on observe là aussi des règles de nommage mais cette-fois en **kebab-case**, c'est-à-dire tout en minuscule avec des tirets.

```
data-compteur-step-value="1"
```

La règle est la suivante : data-[nom-controller]-[nom-value]-value

Étape N°5

Maintenant qu'on a vu comment relier tout ça avec des **normes de nommages** strictes, il ne nous reste plus qu'à dire quoi faire de ces éléments à Stimulus, et c'est là qu'arrive data-action.

La syntaxe est la suivante :

data-action="[événementJS]->[controller]#[méthode]"

Comme par exemple ici :

```
<input
        type="number"
        data-action="change->compteur#changerStep"
        value="1">
```

Ici, on déclenche la méthode **changerStep** du contrôlleur **compteur** lors de l'évènement **change**

4. Aller plus loin

On peut également passer des paramètres à une méthode en les nommant, encore une fois, très strictement comme ceci : data-[controller]-[params]-param=maValeur

```
<button data-action="compteur#changerDe" data-compteur-delta-
param="1">+1</button>
```

Et ça nous permet, dans le contrôleur, d'y avoir accès en récupérant event.params :

```
changerDe(event) {
    // Stimulus extrait les params automatiquement dans event.params
    this.countValue += event.params.delta;
}
```

EXERCICE

- Développez un bouton « reset » qui permet de remettre à zéro la valeur du compteur que l'on a vu dans le cours
- Rajoutez un input HTML contenant la valeur MAXIMALE possible du compteur, par exemple DIX, puis assurez-vous que le compteur ne puisse pas dépasser cette valeur maximale. Attention, si je met 11 dans la valeur maximale du compteur, alors le compteur doit pouvoir atteindre 11 lui aussi !
- Faire la même chose avec une valeur minimale
- Ajoutez une balise <p> vide avec la **target** « message ». Il faut que la balise <p> se remplisse avec un message selon si on a atteint le maximum ou le minimum

- Admettons que ce compteur devienne un compteur de quantité pour un produit à 17.99€. Afficher un prix total à côté sur la page en calculant son prix dynamiquement selon la quantité choisie
- Mettre en place une target « **historique** » qui sera une balise **<ul>**. À chaque opération, ajouter un « li » dans cette **target** pour logger toutes les opérations éffectuées (+1, +4, -3, +7 etc…)